

Exemple de soumission d'activité

ÉTS - LOG430 - Architecture logicielle - Été 2026

Étudiant(e) : Chris-Emmanuel Berton

Questions

(Il est obligatoire d'ajouter du code, des captures d'écran ou des sorties de terminal pour illustrer chacune de vos réponses.)

Question 1 : Dans la RFC 7231, nous trouvons que certaines méthodes HTTP sont considérées comme sûres (safe) ou idempotentes, en fonction de leur capacité à modifier (ou non) l'état de l'application. Lisez les sections 4.2.1 et 4.2.2 de la RFC 7231 et répondez : parmi les méthodes mentionnées dans l'activité 2, lesquelles sont sûres, non sûres, idempotentes et/ou non idempotentes?

Safe -> read only. Exemples : GET, HEAD, OPTIONS, and TRACE Unsafe -> write. Modifie base données. PATCH, CONNECT, POST PUT, DELETE

Idempotentes : Exemple : PUT, DELETE, GET, Non-Idempotent : POST, PATCH and CONNECT.

Les méthodes du test_stock_flow() considérés non sûres et non idempotents sont Post,

```
def test_stock_flow(client):  
    # Création article (`POST /products`)  
    product_data = {'name': 'Some Item', 'sku': '12345', 'price': 99.90}  
    response = client.post('/products',  
                           data=json.dumps(product_data),  
                           content_type='application/json')
```

```
# Write routes (Commands)  
@app.post('/orders')  
def post_orders():  
    """Create a new order based on information on request body"""  
    return create_order(request)
```

alors que les méthodes

sûres sont Get.

```
# Vérification du stock, votre article devra avoir 5 unités dans le stock (`GET /stocks/{product_id}`)  
response = client.get(f'/stocks/{product_id}')  
assert response.status_code == 200  
stock_info = response.get_json()  
assert stock_info['quantity'] == 5
```

```
@app.get('/stocks/<int:product_id>')
def get_stocks(product_id):
    """Get product stocks by product_id"""
    return get_stock(product_id)
```

Finalement, les méthodes Delete et Get sont idempotentes.

```
# Suppression commande et verification du stock post
# Résultat attendu : augmentation.
response = client.delete(f'/orders/{order_id}')
assert response.status_code == 200
assert response.get_json()['deleted'] == True
```

```
@app.delete('/products/<int:product_id>')
def delete_products_id(product_id):
    """Delete a product with a given product_id"""
    return remove_product(product_id)
```

```
# Vérification du stock, votre article devra avoir 5 unités dans le stock (`GET /
response = client.get(f'/stocks/{product_id}')
assert response.status_code == 200
stock_info = response.get_json()
assert stock_info['quantity'] == 5
```

```
@app.get('/stocks/<int:product_id>')
def get_stocks(product_id):
    """Get product stocks by product_id"""
    return get_stock(product_id)
```

2. Question 2 : Décrivez l'utilisation de la méthode join dans ce cas. Utilisez les méthodes telles que décrites à Simple Relationship Joins et Joins to a Target with an ON Clause dans la documentation SQLAlchemy pour ajouter les colonnes demandées dans cette activité. Veuillez inclure le code pour illustrer votre réponse.

La méthode join crée un tableau qui regroupe les informations du stock d'un produit (défini par son id) aux autres informations du produit présents dans la table Product en recherchant les informations associées à l'id du stock mentionné.

```
def get_stock_for_all_products():  
    """Get stock quantity for all products"""  
    session = get_sqlalchemy_session()  
    results = session.query(  
        Stock.product_id,  
        Stock.quantity,  
        Product.name,  
        Product.sku,  
        Product.price  
    ).join(Product, Stock.product_id == Product.id).all()  
    stock_data = []  
    for row in results:  
        stock_data.append({  
            'Article': row.name,  
            'Numéro SKU': row.sku,  
            'Prix unitaire': row.price,  
            'Unités en stock': int(row.quantity),  
        })  
  
    return stock_data
```

3. Question 3 : Quels résultats avez-vous obtenus en utilisant l'endpoint POST /stocks/graphql-query avec la requête suggérée ? Veuillez joindre la sortie de votre requête dans Postman afin d'illustrer votre réponse.

L'endpoint POST /stocks/graphql-query affiche les données associées aux colonnes spécifiées.

none form-data x-www-form-urlencoded raw binary GraphQL Auto Fetch Schema Fetched

QUERY

```

1 {
2   product(id: "1") {
3     id
4     name
5     quantity
6   }
7 }
8

```

GRAPHQL VARIABLES

```

1

```

Body Cookies Headers 5 Test Results 200 OK • 15 ms • 243 B Save Response

{ } JSON Preview Visualize

```

1 {
2   "data": {
3     "product": {
4       "id": 1,
5       "name": "Product 1",
6       "quantity": 56
7     }
8   },
9   "errors": null
10 }
11

```

4. Question 4 : Quelles lignes avez-vous changé dans update_stock_redis? Veuillez joindre du code afin d'illustrer votre réponse.

Les lignes modifiées reflètent les ajouts de colonnes dans la structure de données. De plus, le mapping a été réajusté pour refléter le changement.

```

68 def update_stock_redis(order_items, operation):
69     """ Update stock quantities in Redis """
70     if not order_items:
71         return
72
73     r = get_redis_conn()
74     stock_keys = list(r.scan_iter("stock:"))
75
76     pipeline = r.pipeline()
77     for item in order_items:
78         if hasattr(item, 'product_id'):
79             product_id = item.product_id
80             quantity = item.quantity
81         else:
82             product_id = item['product_id']
83             quantity = item['quantity']
84
85         # TODO: ajoutez plus d'information sur l'article
86         current_stock = r.hget(f"stock:{product_id}", "quantity")
87         current_stock = int(current_stock) if current_stock else 0
88
89         if operation == '+':
90             new_quantity = current_stock + quantity
91         else:
92             new_quantity = current_stock - quantity
93
94         pipeline.hset(f"stock:{product_id}", "quantity", new_quantity)
95
96     pipeline.execute()
97
98 def update_stock_redis(order_items, operation):
99     """ Update stock quantities in Redis """
100    if not order_items:
101        return
102
103    r = get_redis_conn()
104    stock_keys = list(r.scan_iter("stock:"))
105
106    if stock_keys:
107        session = get_sqlalchemy_session()
108        try:
109            product_ids = [
110                item.product_id if hasattr(item, 'product_id') else
111                for item in order_items
112            ]
113            products = session.query(Product).filter(Product.id.in_(product_ids))
114            product_map = {product.id: product for product in products}
115
116            pipeline = r.pipeline()
117            for item in order_items:
118                if hasattr(item, 'product_id'):
119                    product_id = item.product_id
120                    quantity = item.quantity
121                else:
122                    product_id = item['product_id']
123                    quantity = item['quantity']
124
125                current_stock = r.hget(f"stock:{product_id}", "quantity")
126                current_stock = int(current_stock) if current_stock else 0
127
128                if operation == '+':
129                    new_quantity = current_stock + quantity
130                else:
131                    new_quantity = current_stock - quantity
132
133                mapping = {
134                    "quantity": new_quantity
135                }
136                pipeline.hset(f"stock:{product_id}", mapping)
137
138            pipeline.execute()
139

```

```

68 def update_stock_redis(order_items, operation):
74     if stock_keys:
76         for item in order_items:
93-
94- pipeline.execute()
95-
96 else:
97     _populate_redis_from_mysql(r)
98
69 def update_stock_redis(order_items, operation):
102+
103+     mapping = {
104+         "quantity": new_quantity,
105+     }
106+
107+     product = product_map.get(product_id)
108+     if product:
109+         mapping.update({
110+             "name": product.name,
111+             "sku": product.sku,
112+             "price": float(product.price)
113+         })
114+
115+     pipeline.hset(f"stock:{product_id}", mapping=mapping)
116+
117+     pipeline.execute()
118+     finally:
119+         session.close()
120
121 else:
122     _populate_redis_from_mysql(r)

```

5. Question 5 : Quels résultats avez-vous obtenus en utilisant l'endpoint POST /stocks/graphql-query avec les améliorations ? Veuillez joindre la sortie de votre requête dans Postman afin d'illustrer votre réponse.

Quoique POST n'admet pas d'erreurs avec les ajouts, le GraphQL Query ne considère pas être capable de lire, mais répond à la requête adéquatement.

The screenshot shows the Postman interface for a GraphQL query. The query is:

```

2  product(id: "1") {
3    id
4    name
5    quantity
6    sku
7    price
8  }
9

```

The response is a 200 OK status with a JSON body:

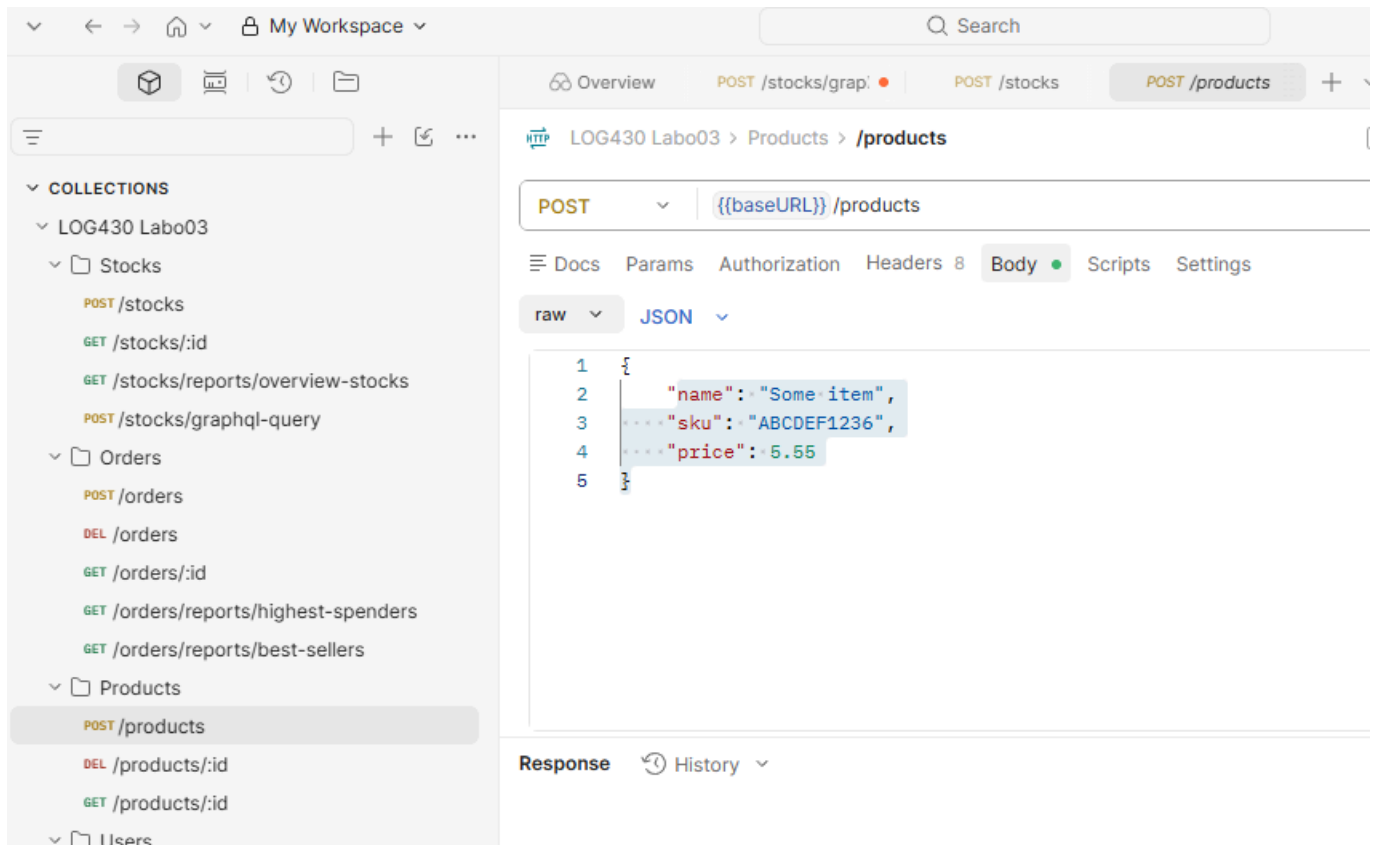
```

1  {
2    "data": {
3      "product": {
4        "id": "1",
5        "name": "Laptop ABC",
6        "price": 1999.99,
7        "quantity": 56,
8        "sku": "LP12567"
9      }
10   },
11   "errors": null
12 }

```

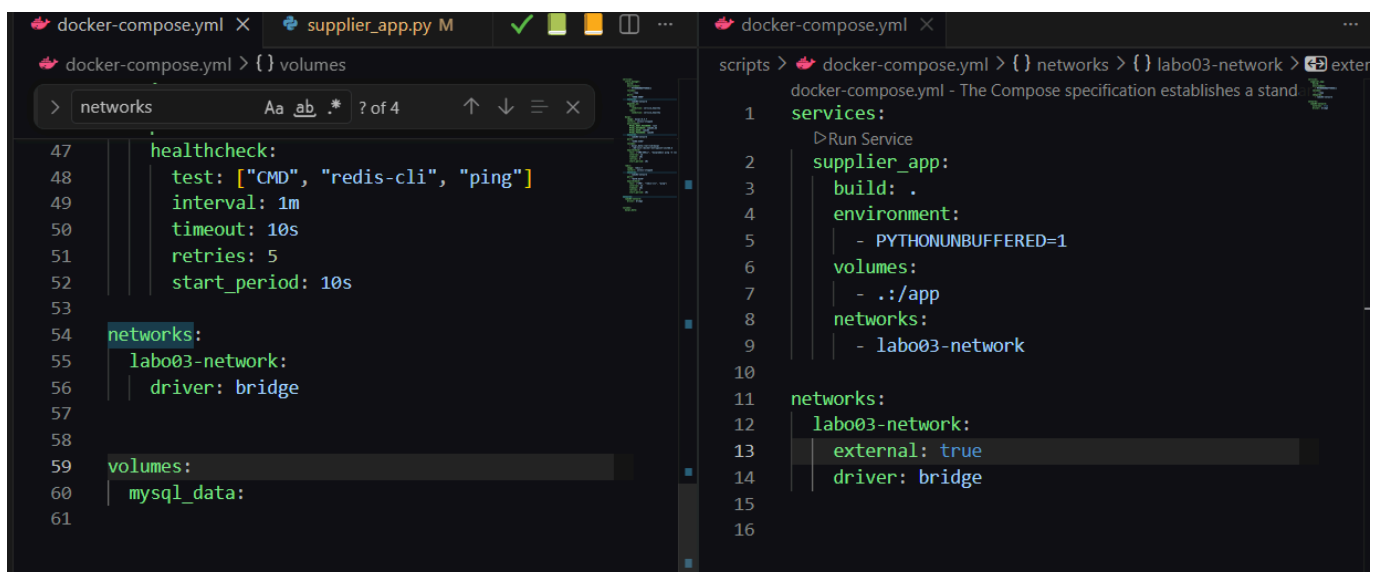
The response also includes a message: "Cannot query field 'price' on type 'Product'.".

En plus, la structure incluant les nouveaux paramètres existe dans le Product.



6. Question 6 : Examinez attentivement le fichier `docker-compose.yml` du répertoire `scripts`, ainsi que celui situé à la racine du projet. Qu'ont-ils en commun ? Par quel mécanisme ces conteneurs peuvent-ils communiquer entre eux ? Veuillez joindre du code YAML afin d'illustrer votre réponse.

Réponse Le point commun entre les `docker-compose` est le `network`. Les deux fichiers spécifient l'utilisation du réseau `labo03-network` et utilisent le driver : `bridge`. La communication se fait à travers le réseau `labo03-network`.



Déploiement

(Le cas échéant, décrivez votre pipeline CI/CD et ce que vous avez appris dans ce laboratoire en ce qui concerne le déploiement. Il est obligatoire d'ajouter du code, des captures d'écran ou des sorties de terminal pour illustrer votre réponse.)

Ce laboratoire a permis de comprendre la connexion entre 2 services en passant par un réseau commun et de communiquer avec un front-end

```
===== 1 failed, 1 passed in 1.92s =====
# pytest
===== test session starts =====
platform linux -- Python 3.11.15, pytest-9.0.3, pluggy-1.6.0
rootdir: /app
configfile: pyproject.toml
collected 2 items

tests/test_store_manager.py ..                                     [100%]

===== 2 passed in 1.41s =====
```

Les configurations de CI/CD sont largement inchangées de la configuration par défaut.

```
.github > workflows > cd.yml > {} jobs > {} deploy > {} steps > {} 2 > abc run
GitHub Workflow - YAML GitHub Workflow (github-workflow.json)
1 name: CD
2 on:
3   push:
4     branches:
5       - main
6 jobs:
7   deploy:
8     runs-on: [self-hosted, linux]
9
10    steps:
11      - name: Checkout
12        uses: actions/checkout@v4
13
14      - name: Create .env file
15        run: |
16          echo "DB_HOST=${{ secrets.DB_HOST }}" >> .env
17          echo "DB_PORT=${{ secrets.DB_PORT }}" >> .env
18          echo "DB_NAME=${{ secrets.DB_NAME }}" >> .env
19          echo "DB_USER=${{ secrets.DB_USER }}" >> .env
20          echo "DB_PASS=${{ secrets.DB_PASS }}" >> .env
21          echo "REDIS_HOST=${{ secrets.REDIS_HOST }}" >> .env
22          echo "REDIS_PORT=${{ secrets.REDIS_PORT }}" >> .env
23          echo "REDIS_DB=${{ secrets.REDIS_DB }}" >> .env
24
25      - name: Restart app
26        run: |
27          docker-compose \
28            -f docker-compose.yml \
29            down
30          docker-compose \
31            -f docker-compose.yml \
32            up -d --build
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

.github > workflows > ci.yml > {} jobs > {} test > {} steps > {} 4 > abc run
GitHub Workflow - YAML GitHub Workflow (github-workflow.json)
1 name: CI
2 on:
3   push:
4     branches: [ main, master ]
5   pull_request:
6
7 jobs:
8   test:
9     runs-on: ubuntu-latest
10    steps:
11      - uses: actions/checkout@v4
12      - name: Set up Python
13        uses: actions/setup-python@v4
14        with:
15          python-version: '3.11'
16      - name: Install dependencies
17        run: |
18          python -m pip install --upgrade pip
19          pip install -r requirements.txt
20
21      - name: Créer fichier .env
22        run: |
23          echo "DB_HOST=127.0.0.1" >> .env
24          echo "DB_PORT=3306" >> .env
25          echo "DB_NAME=labo03_db" >> .env
26          echo "DB_USER=labo03" >> .env
27          echo "DB_PASS=labo03" >> .env
28          echo "REDIS_HOST=127.0.0.1" >> .env
29          echo "REDIS_PORT=6379" >> .env
30          echo "REDIS_DB=0" >> .env
31
32      - name: Run tests
33        working-directory: src
34        run: echo "Call pytest command here!"
```

Le changement majeur est la configuration réseau des docker-compose (voir image Question 6).

Au premier lancement de la config présente dans le commit Task 6 : Done. TODO : extra, la communication entre supplier_app.py et store_manager fonctionnait.

```
supplier_app-1 | 2026-06-04 22:34:02,396 - ERROR - Connection error on attempt 1
supplier_app-1 | 2026-06-04 22:34:04,397 - INFO
- Calling http://store_manager:5000/stocks/graphql-query (attempt 2/3)
supplier_app-1 | 2026-06-04 22:34:08,416 - INFO - Response: 200 - OK
supplier_app-1 | 2026-06-04 22:34:08,416 - INFO - Response body: {"data":
{"product":{"id":1,"name":"Laptop ABC","quantity":56}}, "errors":null}
```



```
supplier_app-1 | ... supplier_app-1 | 2026-06-04 22:34:08,417
- INFO - Waiting 10 seconds until next call... supplier_app-1 | 2026-06-04
22:34:18,418
- INFO - --- Call #2 --- supplier_app-1 | 2026-06-04 22:34:18,420
- INFO - Calling http://store_manager:5000/stocks/graphql-query (attempt 1/3)
supplier_app-1 | 2026-06-04 22:34:18,431
- INFO - Response: 200 - OK supplier_app-1 | 2026-06-04 22:34:18,432 - INFO -
Response body: {"data":{"product":{"id":1,"name":"Laptop
ABC","quantity":56}},"errors":null} supplier_app-1 | ... supplier_app-1 | 2026-06-
04 22:34:18,432
- INFO - Waiting 10 seconds until next call... supplier_app-1 | 2026-06-04
22:34:28,431 - INFO - --- Call #3 --- supplier_app-1 | 2026-06-04 22:34:28,433
- INFO - Calling http://store_manager:5000/stocks/graphql-query (attempt 1/3)
supplier_app-1 | 2026-06-04 22:34:28,446
- INFO - Response: 200 - OK supplier_app-1 | 2026-06-04 22:34:28,447 - INFO -
Response body: {"data":{"product":{"id":1,"name":"Laptop
ABC","quantity":56}},"errors":null} supplier_app-1 | ... supplier_app-1 | 2026-06-
04 22:34:28,447
- INFO - Waiting 10 seconds until next call... supplier_app-1 | 2026-06-04
22:34:38,449 - INFO - --- Call #4 --- supplier_app-1 | 2026-06-04 22:34:38,450 -
INFO - Calling http://store_manager:5000/stocks/graphql-query (attempt 1/3)
supplier_app-1 | 2026-06-04 22:34:38,458
- INFO - Response: 200 - OK supplier_app-1 | 2026-06-04 22:34:38,459 - INFO -
Response body: {"data":{"product":{"id":1,"name":"Laptop
ABC","quantity":56}},"errors":null} supplier_app-1 | ... supplier_app-1 | 2026-06-
04 22:34:38,459
- INFO - Waiting 10 seconds until next call... supplier_app-1 | 2026-06-04
22:34:48,460
- INFO - --- Call #5 --- supplier_app-1 | 2026-06-04 22:34:48,461
- INFO - Calling http://store_manager:5000/stocks/graphql-query (attempt 1/3)
supplier_app-1 | 2026-06-04 22:34:48,469
- INFO - Response: 200 - OK supplier_app-1 | 2026-06-04 22:34:48,470 - INFO -
Response body: {"data":{"product":{"id":1,"name":"Laptop
ABC","quantity":56}},"errors":null} supplier_app-1 | ... supplier_app-1 | 2026-06-
04 22:34:48,472
- INFO - Waiting 10 seconds until next call... supplier_app-1 | 2026-06-04
22:34:58,472
- INFO - --- Call #6 --- supplier_app-1 | 2026-06-04 22:34:58,473
- INFO - Calling http://store_manager:5000/stocks/graphql-query (attempt 1/3)
supplier_app-1 | 2026-06-04 22:34:58,481
- INFO - Response: 200 - OK supplier_app-1 | 2026-06-04 22:34:58,482
- INFO - Response body: {"data":{"product":{"id":1,"name":"Laptop
ABC","quantity":56}},"errors":null}
```

Cependant, les builds subséquents de cette même config échouaient.


```
2026-06-05 00:37:27,533 - INFO - Starting periodic calls to http://store-manager:5000/stocks/graphql-query every 10 seconds
2026-06-05 00:37:27,534 - INFO - Press Ctrl+C to stop
2026-06-05 00:37:27,534 - INFO - --- Call #1 ---
2026-06-05 00:37:27,538 - INFO - Calling http://store-manager:5000/stocks/graphql-query (attempt 1/3)
2026-06-05 00:37:35,549 - ERROR - Connection error on attempt 1
2026-06-05 00:37:37,550 - INFO - Calling http://store-manager:5000/stocks/graphql-query (attempt 2/3)
2026-06-05 00:37:45,563 - ERROR - Connection error on attempt 2
2026-06-05 00:37:47,564 - INFO - Calling http://store-manager:5000/stocks/graphql-query (attempt 3/3)
2026-06-05 00:37:55,569 - ERROR - Connection error on attempt 3
2026-06-05 00:37:55,578 - ERROR - All 3 attempts failed for http://store-manager:5000/stocks/graphql-query
2026-06-05 00:37:55,580 - INFO - Waiting 10 seconds until next call...
2026-06-05 00:38:05,588 - INFO - --- Call #2 ---
2026-06-05 00:38:05,589 - INFO - Calling http://store-manager:5000/stocks/graphql-query (attempt 1/3)
2026-06-05 00:38:13,594 - ERROR - Connection error on attempt 1
2026-06-05 00:38:15,595 - INFO - Calling http://store-manager:5000/stocks/graphql-query (attempt 2/3)
2026-06-05 00:38:23,600 - ERROR - Connection error on attempt 2
2026-06-05 00:38:25,602 - INFO - Calling http://store-manager:5000/stocks/graphql-query (attempt 3/3)
2026-06-05 00:38:33,607 - ERROR - Connection error on attempt 3
2026-06-05 00:38:33,609 - ERROR - All 3 attempts failed for http://store-manager:5000/stocks/graphql-query
2026-06-05 00:38:33,612 - INFO - Waiting 10 seconds until next call...
2026-06-05 00:38:43,614 - INFO - --- Call #3 ---
2026-06-05 00:38:43,614 - INFO - Calling http://store-manager:5000/stocks/graphql-query (attempt 1/3)
2026-06-05 00:38:51,618 - ERROR - Connection error on attempt 1
2026-06-05 00:38:53,620 - INFO - Calling http://store-manager:5000/stocks/graphql-query (attempt 2/3)
2026-06-05 00:39:01,623 - ERROR - Connection error on attempt 2
2026-06-05 00:39:03,625 - INFO - Calling http://store-manager:5000/stocks/graphql-query (attempt 3/3)
2026-06-05 00:39:11,629 - ERROR - Connection error on attempt 3
2026-06-05 00:39:11,630 - ERROR - All 3 attempts failed for http://store-manager:5000/stocks/graphql-query
2026-06-05 00:39:11,630 - INFO - Waiting 10 seconds until next call...
2026-06-05 00:39:21,631 - INFO - --- Call #4 ---
2026-06-05 00:39:21,632 - INFO - Calling http://store-manager:5000/stocks/graphql-query (attempt 1/3)
2026-06-05 00:39:29,634 - ERROR - Connection error on attempt 1
2026-06-05 00:39:31,635 - INFO - Calling http://store-manager:5000/stocks/graphql-query (attempt 2/3)
```

2026-06-05 00:39:31,635 - INFO - Calling <http://store-manager:5000/stocks/graphql-query> (attempt 2/3)
2026-06-05 00:39:39,641 - ERROR - Connection error on attempt 2
2026-06-05 00:39:41,642 - INFO - Calling <http://store-manager:5000/stocks/graphql-query> (attempt 3/3)
2026-06-05 00:39:49,646 - ERROR - Connection error on attempt 3
2026-06-05 00:39:49,646 - ERROR - All 3 attempts failed for <http://store-manager:5000/stocks/graphql-query>
2026-06-05 00:39:49,647 - INFO - Waiting 10 seconds until next call...
2026-06-05 00:39:59,646 - INFO - --- Call #5 ---
2026-06-05 00:39:59,646 - INFO - Calling <http://store-manager:5000/stocks/graphql-query> (attempt 1/3)
2026-06-05 00:40:07,651 - ERROR - Connection error on attempt 1
2026-06-05 00:40:09,652 - INFO - Calling <http://store-manager:5000/stocks/graphql-query> (attempt 2/3)
2026-06-05 00:40:17,656 - ERROR - Connection error on attempt 2
2026-06-05 00:40:19,657 - INFO - Calling <http://store-manager:5000/stocks/graphql-query> (attempt 3/3)
2026-06-05 00:40:27,659 - ERROR - Connection error on attempt 3
2026-06-05 00:40:27,659 - ERROR - All 3 attempts failed for <http://store-manager:5000/stocks/graphql-query>
2026-06-05 00:40:27,661 - INFO - Waiting 10 seconds until next call...
2026-06-05 00:40:37,664 - INFO - --- Call #6 ---
2026-06-05 00:40:37,665 - INFO - Calling <http://store-manager:5000/stocks/graphql-query> (attempt 1/3)
2026-06-05 00:40:45,680 - ERROR - Connection error on attempt 1
2026-06-05 00:40:47,680 - INFO - Calling <http://store-manager:5000/stocks/graphql-query> (attempt 2/3)
2026-06-05 00:40:55,685 - ERROR - Connection error on attempt 2
2026-06-05 00:40:57,684 - INFO - Calling <http://store-manager:5000/stocks/graphql-query> (attempt 3/3)
2026-06-05 00:41:05,688 - ERROR - Connection error on attempt 3
2026-06-05 00:41:05,689 - ERROR - All 3 attempts failed for <http://store-manager:5000/stocks/graphql-query>
2026-06-05 00:41:05,690 - INFO - Waiting 10 seconds until next call...
2026-06-05 00:41:15,691 - INFO - --- Call #7 ---
2026-06-05 00:41:15,692 - INFO - Calling <http://store-manager:5000/stocks/graphql-query> (attempt 1/3)
2026-06-05 00:41:23,697 - ERROR - Connection error on attempt 1
2026-06-05 00:41:25,698 - INFO - Calling <http://store-manager:5000/stocks/graphql-query> (attempt 2/3)
2026-06-05 00:41:33,703 - ERROR - Connection error on attempt 2
2026-06-05 00:41:35,704 - INFO - Calling <http://store-manager:5000/stocks/graphql-query> (attempt 3/3)
2026-06-05 00:41:43,710 - ERROR - Connection error on attempt 3
2026-06-05 00:41:43,710 - ERROR - All 3 attempts failed for <http://store-manager:5000/stocks/graphql-query>
2026-06-05 00:41:43,711 - INFO - Waiting 10 seconds until next call...